

Investigating Fourier Representations and Learned Algorithms within Grokked Transformers

Robert Shin

August 2024

Abstract

Grokking is a form of emergent behavior where a small model trained on an algorithmic task achieves a delayed generalization long after originally overfitting. Grokking was first reported in small transformer models trained on algorithmic tasks, but recent research points to a ubiquity of delayed generalizations throughout deep networks. The first goal of this experiment is to document how grokking behavior scales in 1-layer transformers on tasks of increasing complexity. Defining problem complexity is not straightforward however, thus linking it to emergent behavior remains a challenge. The secondary goal of the project is to interpret the learned algorithms of the models by projecting its embeddings and activations into the Fourier domain. In doing so, it becomes clear that the model learns to solve modular arithmetic questions with a “clock” algorithm, cleverly using trigonometric identities to calculate modular sums. By analyzing how the model develops its circuits to respond to certain key frequency trigonometric functions, I show that the delayed generalization is a result of the model’s components working in the same frequencies to calculate sums. Attention analysis on a model trained to do both modular addition and modular subtraction show that the grokked model develops asymmetries in its attention mechanism allowing it to switch between operations.

1. Introduction

Grokking is a form of emergent behavior where a small model trained on an algorithmic task achieves generalization long after originally overfitting (Power et al., 2022; Wei et al., 2022). After quickly reaching 100% training accuracy, the model takes several thousand iterations before its testing accuracy slowly reaches 100% as well. This strange phenomenon can be understood through various *progress measures*, proving that the model is slowly developing various interpretable circuits after overfitting (Nanda et al., 2023). This project began with recreating the grokking phenomenon on different algorithmic tasks, specifically modular arithmetic, modular subtraction, and joint modular arithmetic. The first goal was to investigate how the timing of the phase change scaled with the complexity of the task. Past research has focused on predicting grokking by looking at the loss landscape of early iterations, characterized by oscillatory behaviors in training curves (Notsawo et al., 2023).

Unlike in the fields of physics and chemistry, phase changes in neural networks are not well defined by physical laws or theories (Wei et al., 2022). Developing a theory of when emergent behaviors occur could aid in predicting when models could reach AGI or other unforeseen behavior. Recent findings have shown that grokking is a common occurrence in deep networks and may be necessary for transformers to develop capacities like implicit reasoning (Humayun et al., 2024; Wang et al., 2024). This project is therefore interested in exploring grokking behavior as well as interpreting the algorithms learned through the grokking process. Though the model in this experiment is small (1-layer transformer), it may be possible to interpret behavior in small circuits within larger models in the future (Nanda et al., 2024).

Grokking on a given task does not always lead to the same learned algorithm, due to changes in the model’s parameters (Zhong et al., 2023). The model used in this paper experienced grokking under certain conditions, ultimately learning the “clock” algorithm described by Nanda et al. This is proven by showing that the model uses trigonometric functions in a number of key frequencies that allow it to perform modular arithmetic. The model is later trained to perform modular subtraction as well as joint modular arithmetic (addition and subtraction where the final input token indicates the operation) where it exhibits controlled asymmetries in its attention mechanism.

2. Background & Setup

The grokking phenomenon was first reported in Powers et al. where they found surprising train/test loss behavior in 2-layer transformers trained on algorithmic data. These transformers were shown to overfit quickly during the training process while remaining ineffective on test data, then suddenly transition to perfect generalization on the dataset long after overfitting. Later work by Nanda et al. proves the exact algorithm – referred to as the Fourier multiplication or “clock” algorithm – a 1-layer transformer uses in the case of modular addition. The experimental setup in this paper is meant to mimic Nanda’s, with the goal of recreating the clock algorithm and interpreting a similar model trained on addition and subtraction.

The task: $a \circ b \bmod P = ?$

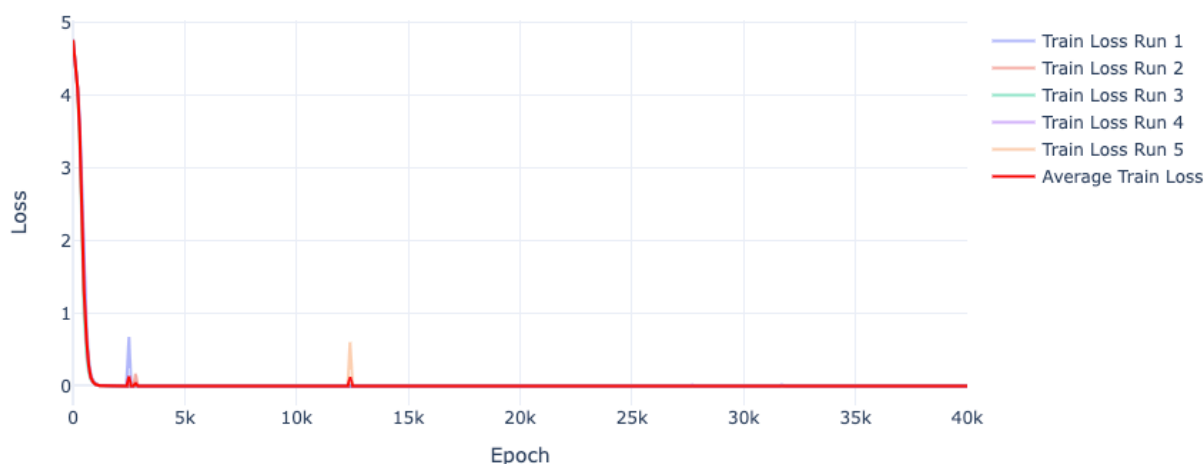
The model used in this experiment is a 1-layer transformer trained on modular arithmetic with modulo $P = 113$. Data is fed into the model in sets of three tokens – the first containing the integer a , the second containing the integer b , and the last token varied between experiments. In the cases of modular addition and subtraction, the final token was always P , remaining constant throughout training. When the transformer was trained on both addition and subtraction, the final token was P for addition and $P+1$ for subtraction. This allowed the model to learn to differentiate between the operations given the last token. Each datapoint was labeled with its correct modular sum.

Data was generated by computing each of the $P^2 = 12769$ pairs of modular sums $a, b < P$. This gives $113+1$ unique tokens (One extra for the last token P) leading to an embedding matrix of size $(114, d=128)$. The model also develops learned positional embeddings for the three tokens. The attention layer is split into 4 heads of size $d/4 = 32$, with $n = 512$ neurons in the MLP. The model uses a ReLU activation and a final LayerNorm before unembedding into 114 logits. The individual attention layer does not use LayerNorm, embed/unembed matrices are not tied and there are no biases used anywhere in the model.

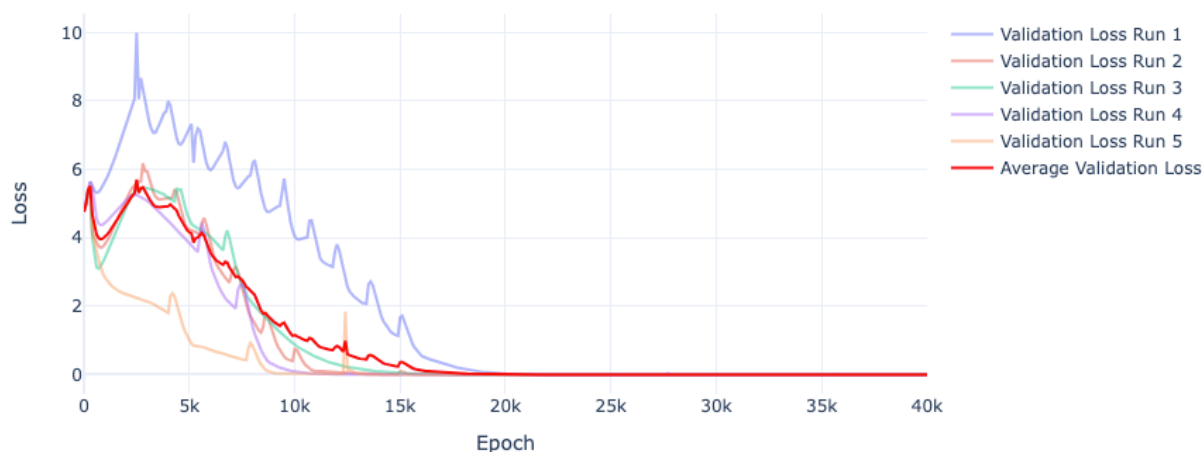
In order to induce grokking behavior, a randomly chosen $r = 30\%$ of the P^2 pairs are used for training while the rest are used for validation. The joint arithmetic model was trained with different train/test splits including $r = 16.3\%$. Prior grokking research has shown that higher train/test ratios have led to generalization without grokking and models trained on lower ratios never generalize at all (Power et al., 2022). The model is trained on full batch gradient descent using the AdamW optimizer with learning rates $\gamma=0.0002, 0.001$ and weight decay $\lambda = 1$. The high weight decay is a necessary feature for the model to change after overfitting and eventually grok (Power et al., 2022).

Given these conditions, the model consistently exhibits grokking behavior. Pictured below are the train and test loss curves for modular addition over 5 different seeds. There was one 6th seed that exhibited unexpected behavior which will be discussed later on. The validation loss exhibits inconsistent and “spiky” progress, occasionally dropping out of loss minima in certain locations.

Training Loss for Multiple Runs

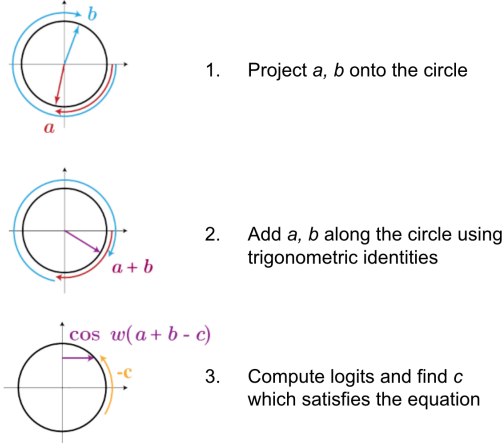


Validation Loss for Multiple Runs



3. The “Clock” Algorithm

The method discovered by Nanda et al. involves projecting the models inputs into sinusoidal functions and using a number of clever trigonometric identities to achieve modular addition. Adding time on a clock is an intuitive lens to view modular addition. 3 hours past 10 a.m. is 1 p.m., not 13 a.m. – we have achieved addition mod 12. The transformer implements something similar, projecting numbers into points along a circle and adding them along a circle with a period of 113 rather than 12.



(Nanda et al., 2022)

A more in-depth explanation of the algorithm can be found in Nanda's original paper, but here is a brief description: The model first projects its inputs a, b into $\sin(w_k a), \cos(w_k a), \sin(w_k b), \cos(w_k b)$ where $w_k = 2k\pi/P$ for a number of key frequencies using its embedding matrix. The model then uses attention and ReLU to calculate quadratic terms and use the following trigonometric identity:

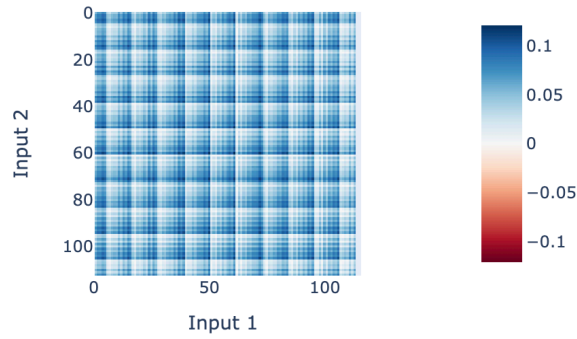
$$\begin{aligned}\cos(w_k(a+b)) &= \cos(w_k a)\cos(w_k b) - \sin(w_k a)\sin(w_k b) \\ \sin(w_k(a+b)) &= \sin(w_k a)\cos(w_k b) + \cos(w_k a)\sin(w_k b)\end{aligned}$$

The model then calculates $\cos(w_k(a+b-c))$ for each output logit c using the same identity as above. The model adds the value $\cos(w_k(a+b-c))$ at each logit for each key frequency in its unembedding matrix. This causes constructive interference at the logit c where cosine is maximized and destructive interference elsewhere. Maximizing cosine in this case leads to finding the correct solution to the modular arithmetic problem.

$$\begin{aligned}w_k(a+b-c) &= 0 \Rightarrow c = a+b \\ w_k(a+b-c) &= 2k\pi \Rightarrow c = a+b-P\end{aligned}$$

Aside: Intuition on Periodicity and Sparsity in Fourier Domain

Many of the methods included in this project and its inspirations are based on the observation of periodicity in the model's various activations. Pictured below is the activation of the first neuron in the MLP where inputs 1 and 2 represent a and b . The activations make a quilt-like pattern – clearly the neuron responds to inputs in a periodic way.



This neuron is one of many examples of the model expressing periodicity in its features – but what does that imply about the model? Perhaps the current basis isn’t the best way to express the model’s activations. The key intuition is that functions that are periodic in the regular domain appear to be “sparse” in the Fourier domain. The Fourier Transform represents functions as sums of sines and cosines, so it follows that a periodic function could be expressed in fewer waves than a non-periodic function.

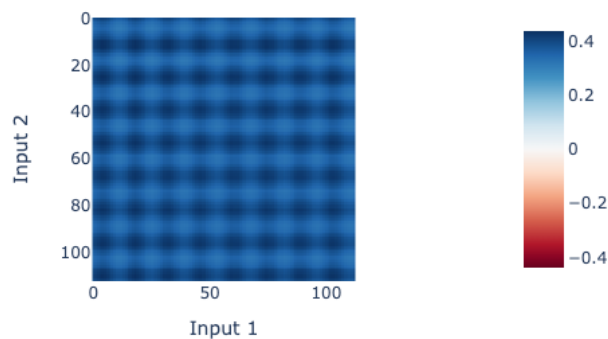


This implies that our model may be choosing to work in trigonometric functions of a few select frequencies.

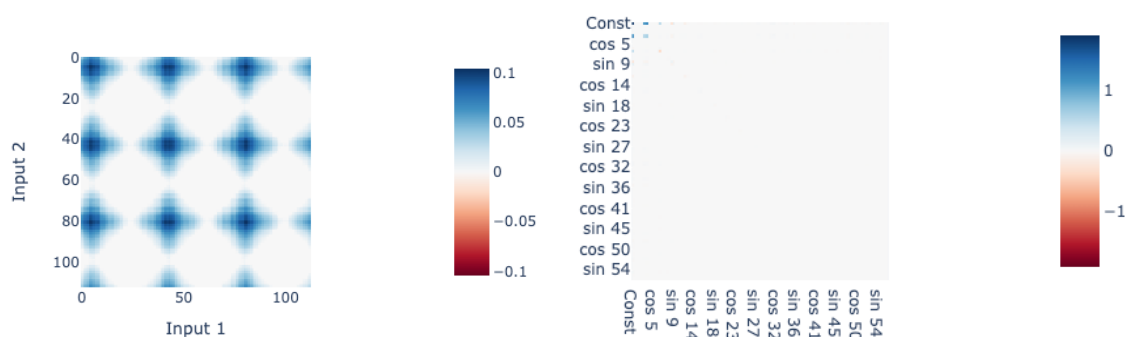
4. Methods

4.1 Observing Periodic Activations

Due to the relatively small input space $P = 113$, it is possible to visualize model activations as a 2-D function of a and b for all P^2 possible inputs. These periodic activations appear in the model’s attention scores as well as individual neuron activations. When the activations are converted into the Fourier basis they are extremely sparse. Pictured below are the first attention head’s activations calculated at the first position in the sequence.



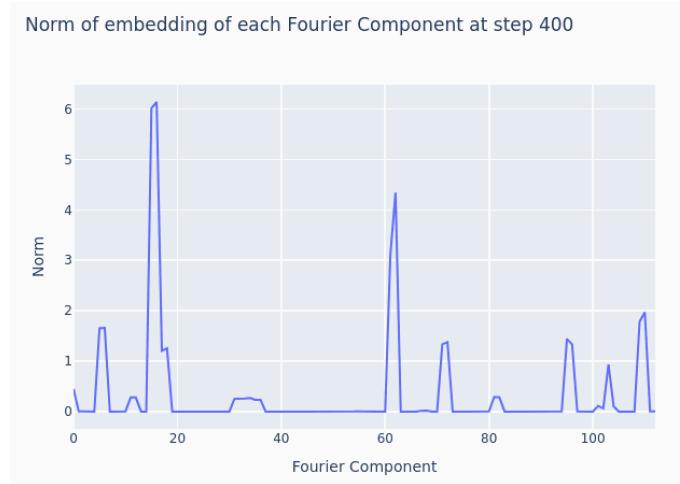
Here is a neuron with particularly periodic activations alongside its sparse representation in the Fourier basis.



4.2 Identifying Key Frequencies

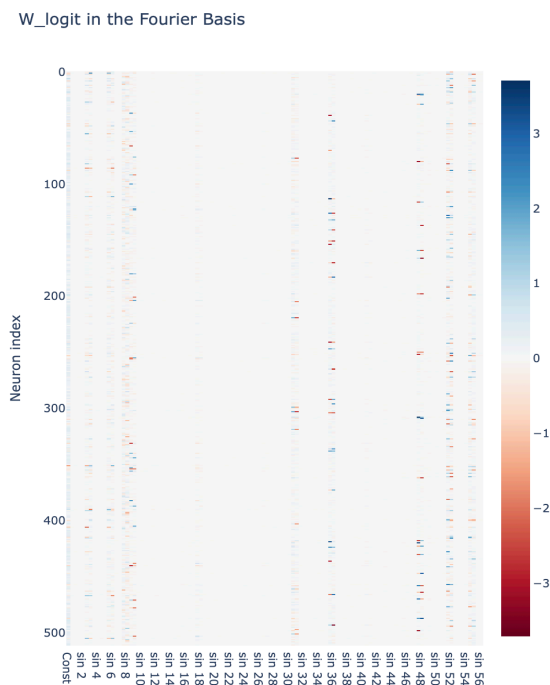
As demonstrated by the neuron activations above, the model tends to rely on a number of key frequencies. These frequencies are consistent through each layer of the model and the exact key frequencies vary between training runs. These frequencies can first be identified by projecting the model's embedding matrix into the Fourier basis. The Fourier embedding matrix can also be used as a qualitative *progress measure* proving that the model is slowly developing productive circuits during the grokking process. While the phase change to perfect generalization may seem random and abrupt, the Fourier embedding shows that the generalization is a result of the model correctly aligning to key frequencies across its layers.

Pictured below is the embedding matrix of the transformer that has grokked modular addition after 40,000 epochs. A number of key frequencies are favored while the others are nullified. An important note is that the x-axis includes sines and cosines next to each other. This explains why the peaks appear to have two edges on top – those are the sine and cosine values at the same key frequency. So the first peak refers to $\cos(8)$ and $\sin(8)$ beside it.



These key frequencies are not limited to the embedding matrix however. By tracking the squared activations of individual neurons to waves of certain frequencies, we can calculate the fraction of variance explained by a given frequency. This means that neurons tend to fluctuate in response to certain frequencies more than others, implying that they have learned to react to those key frequencies. In analyzing key frequencies for each neuron, we find that they match the key frequencies of the embedding matrix. However, we'll find that these neurons are not as well explained by these frequencies as they were in past experiments, and that ablations and restricted loss do not give valid progress measures to this model.

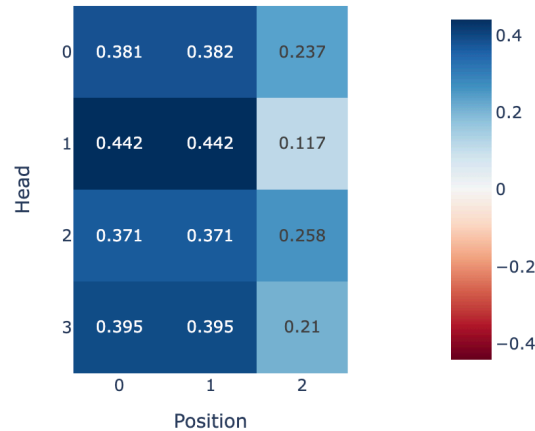
Lastly, by multiplying the neuron outputs with the final layernorm and the unembed matrix (W_{logit}), output logits once again favor the same key frequencies in the Fourier domain. Shown below, we see that the faint activation stripes occur at the same key frequencies as before.



4.3 Attention Scores by Position

After calculating the attention score for all P^2 inputs at each position, we compare how the model gives attention scores on average depending on the position. Below are the ratios of average attention scores for each position among the 4 attention heads of the transformer trained on modular addition.

Attention score for heads at each position



We can see that each head gives equal attention scores to the first two positions which refer to a and b . This result is expected as addition is commutative and therefore the order of the first two tokens shouldn't be important. We later see that the models trained on subtraction and joint arithmetic divide attention scores asymmetrically.

With these methods I first analyze grokking behavior in the models and then move onto interpreting the models' learned algorithms.

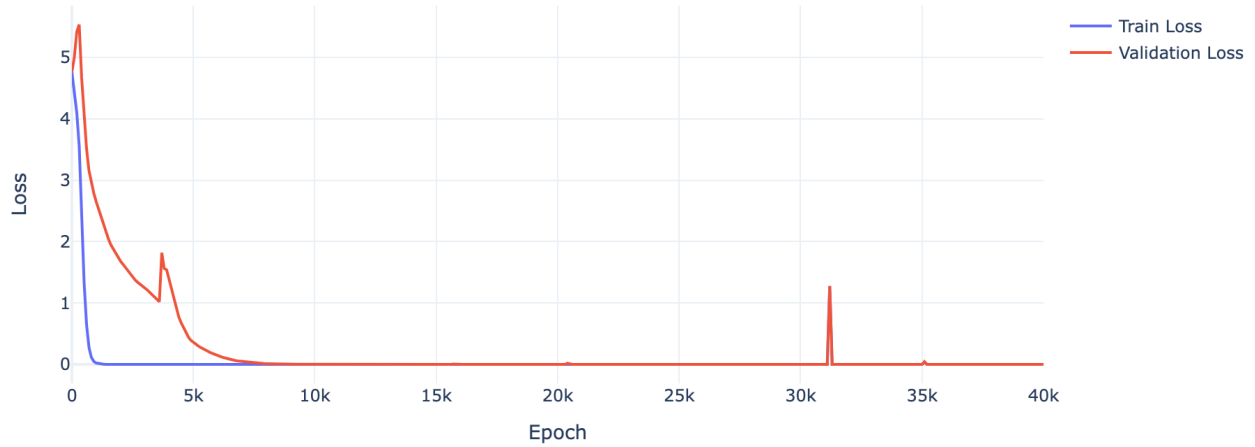
5. Results and Analysis

5.1 Grokking Progress

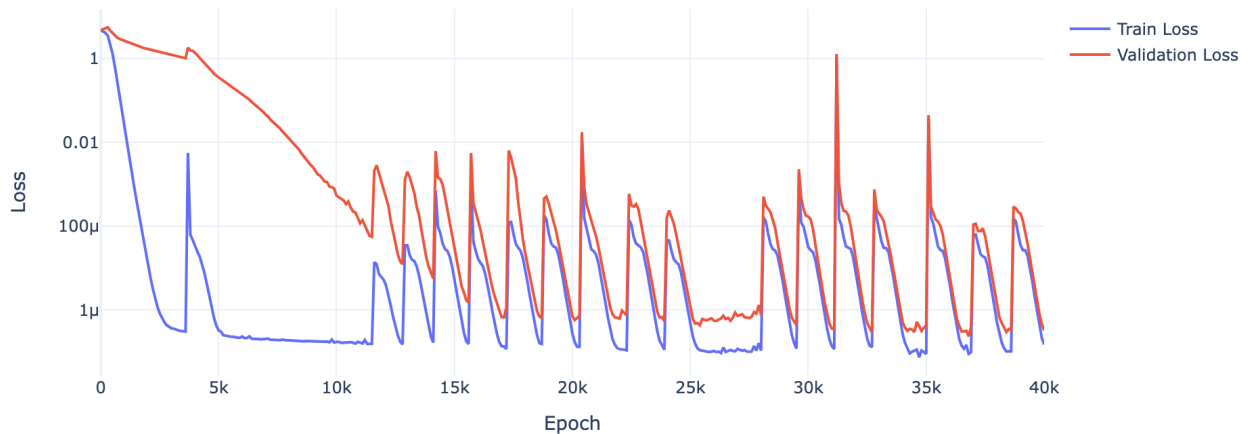
The first step of this experiment was to recreate the grokking phenomenon on my own transformer model. Despite using the same data format, layering, and hyperparameters, my model would not exhibit grokking. It wouldn't complete the memorization step (overfitting), and never generalized on testing data. It was found that by adding a final LayerNorm after the MLP, the model was able to grok with the same behavior as past experiments. While grokking was successfully achieved, this points to a fundamental difference between my model and those in past experiments which I was unable to pinpoint and correct.

This difference actualizes itself in the poor stability of the model. Pictured are the normal and log-scaled loss curves of a model trained on modular arithmetic.

Grokking Training Curve



Grokking Training Curve (log-scaled)

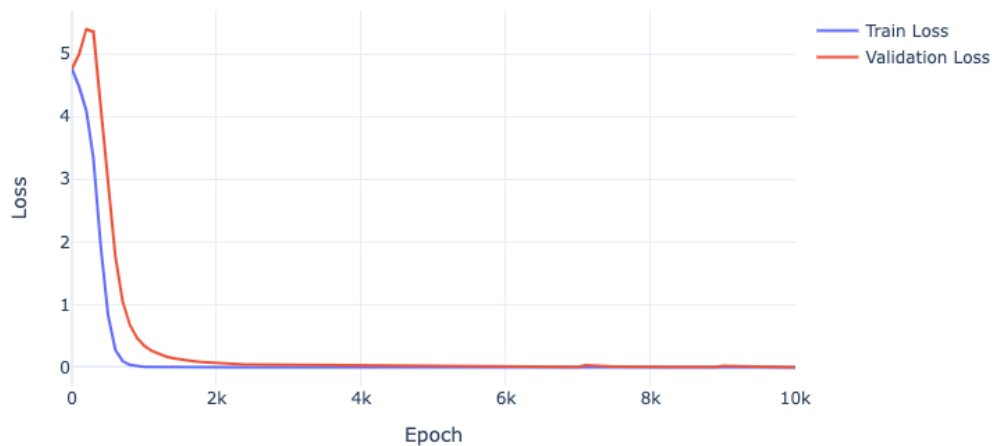


Past grokking experiments used a learning rate of 0.001 without decay, but did not experience the same spikes in both train and test loss (Nanda et al., 2023). The model used in this experiment appeared to be much less stable even when trained on longer durations. For this reason, a $\gamma = 0.0002$ was used for future runs. However, it was later noted that the oscillations in training and testing behavior is characteristic in predicting future grokking (Notsawo et al., 2023).

The same experimental setup was tested for six different seeds determining how initial weight matrices are set and how train/test data is split. On five of the six runs, the model groks as expected, and those five runs can be seen in the very first figure including an average of the five runs. However, on seed 1340, the model generalizes much more quickly than expected, though never achieves a final cross-entropy loss as low as other runs. This model develops its key frequencies much more quickly than other runs, possibly as a random byproduct of how its weights were initialized. Observing seed 1340's training curve below, the model already begins to generalize on test data before overfitting is finished. This model appears to generalize without the characteristic grokking pattern. The relatively high final cross-entropy loss may imply that grokking leads to

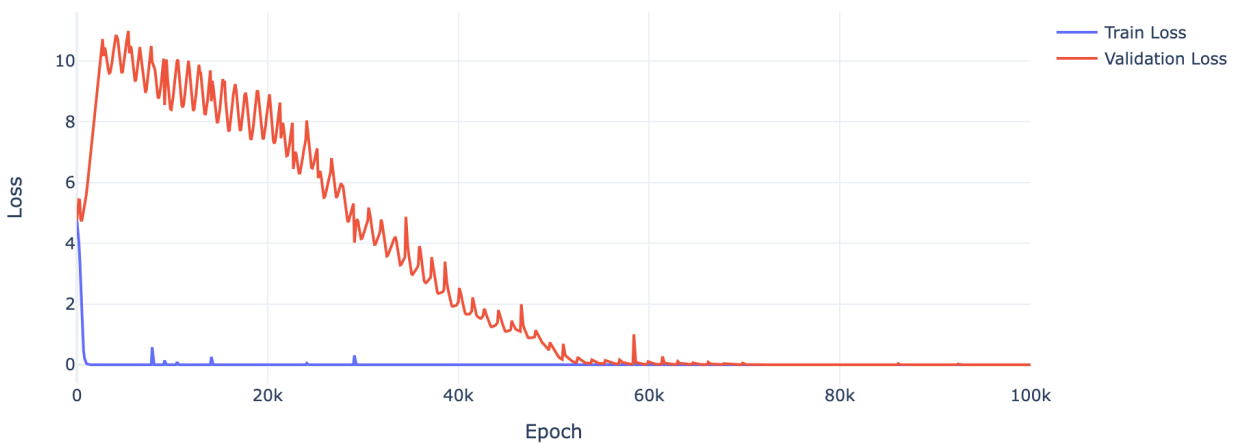
more accurate generalization. Also, the model trained on this seed does not generalize exceptionally quickly on the joint arithmetic case as it did for modular addition alone.

Grokking Training Curve



After confirming the model's mostly consistent ability to grok modular addition, the model was then trained on modular subtraction data with the same hyperparameters.

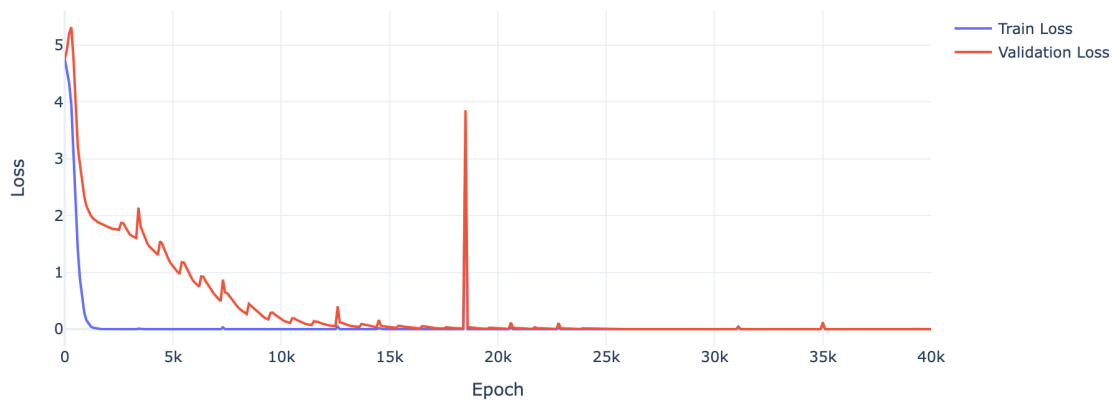
Grokking Training Curve (subtraction)



Note that this model took 60k epochs to generalize on modular subtraction, which is much longer than modular addition on average. The test loss dropoff is also much slower and more consistent than for addition, seeming less like a phase change and more like slow progress. It is surprising that subtraction would pose such a challenge compared to addition, but perhaps it is the non-commutative property that confuses the model – it must learn to use the first two tokens differently whereas addition was more lenient.

The final task that the model was trained on was both addition and subtraction, where the final token indicates which operation to choose. In order to train this model, addition and subtraction data were generated over the 113^2 inputs and added together. Initially, the same fraction of data ($r = 30\%$) was chosen for training as before. This resulted in the model learning much more quickly than it did for subtraction.

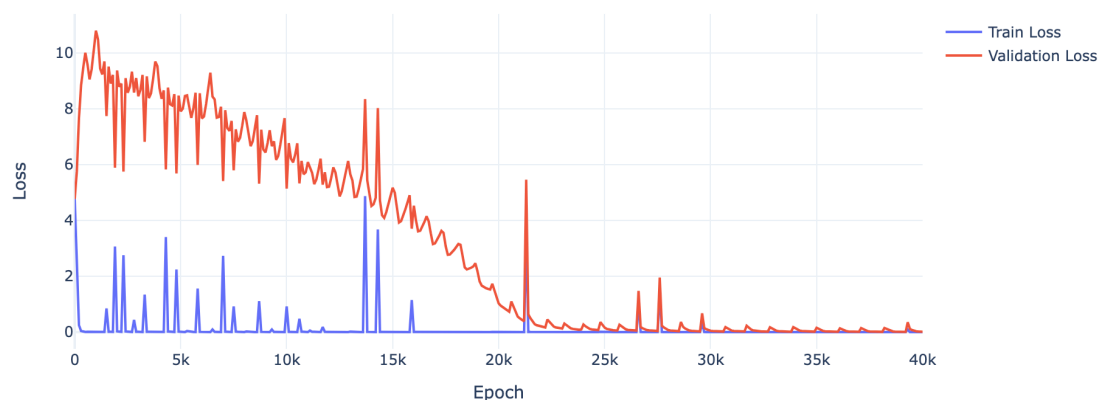
Grokking Training Curve (joint arithmetic)



It's a bit odd that the model can learn to differentiate and solve two separate tasks more quickly than it can learn just one of those tasks. There are two plausible explanations for this. First, that learning addition helps the model develop circuits that respond to key frequencies quickly, accelerating its ability to learn subtraction as well. This is supported by recent findings where it was found that using the embedding matrix from a model that grokked modular addition sped up a new model's ability to learn modular subtraction (Furuta et al., 2023). To explore this, the model's loss metrics on addition and subtraction were tracked separately during training. The model's accuracy on addition tended to advance slightly more quickly than on subtraction, though this is more likely because addition is easier to learn in the first place.

The second explanation for the model's ability to learn so quickly is that the model is seeing more data. During training it sees 30% of addition pairs and 30% of subtraction pairs, which only differ in the last token. It's expected to see $\sim 51\%$ of possible pairs, giving it an advantage over the subtraction case. To scale back to an expected 30% of data seen, I set $r = 16.3\%$ for future training runs. However, with $\gamma = 0.0002$, the model failed to generalize at all. Putting the learning rate back to 0.001 resulted in this "spiky" graph once again.

Grokking Training Curve (joint arithmetic)

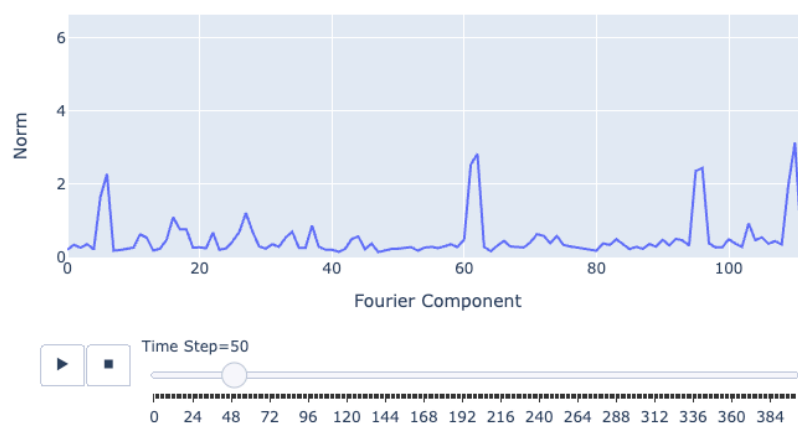


Given the train/test split for this model as well as the different learning rate, it's difficult to make conclusions on how grokking behavior scales with problem complexity. I also trained the model on modular multiplication (more information post-conclusion) and it generalized even more quickly than in the case of addition. Although the grokking “phase change” can be explained through consistent progress measures, the duration it takes the model to grok is not easily interpretable. Certain algorithmic tasks like modular multiplication may seem more involved than modular addition to us, but we cannot quantitatively predict how difficult they will be for a transformer to grok.

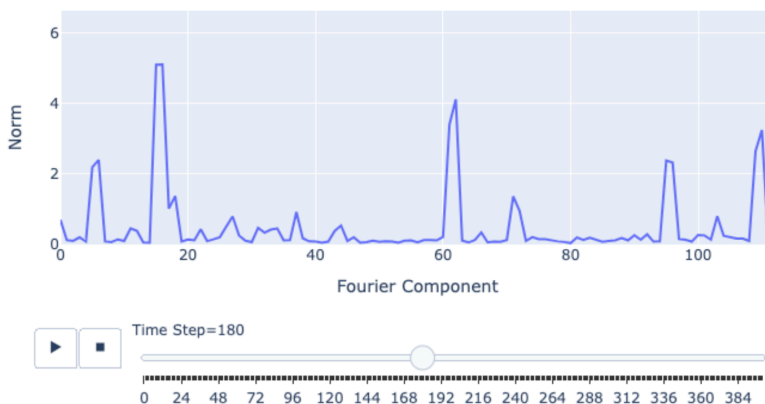
5.2 Reverse-Engineering the Models’ Learned Algorithms

The second step of the experiment was to reverse engineer the clock algorithm in my own model, then see what algorithm the transformer learns in the other modular tasks. After successfully observing periodicity in both attention head and neuron activations, I projected the model’s embedding matrix into the Fourier basis. Below are the addition model’s Fourier embeddings after epoch 5,000, 18,000, and 40,000 (each step is 100 epochs). The last embedding matrix is fully trained, and this model was extremely accurate with a final cross-entropy loss of 9.7×10^{-8} .

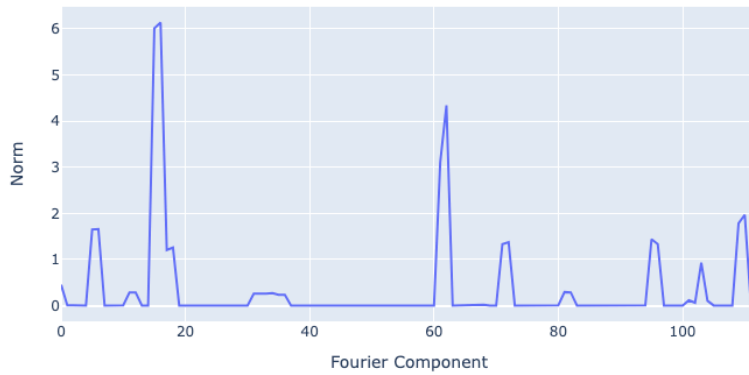
Evolution of Embedding Norm Over Time



Evolution of Embedding Norm Over Time

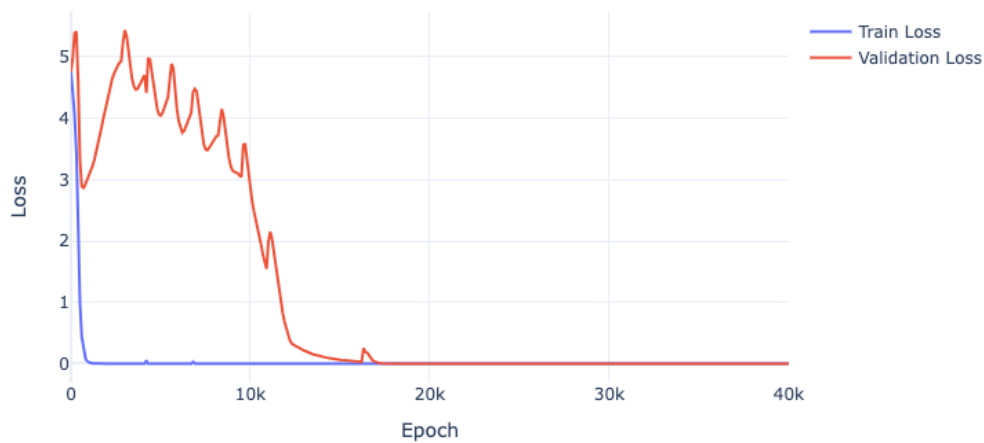


Norm of embedding of each Fourier Component

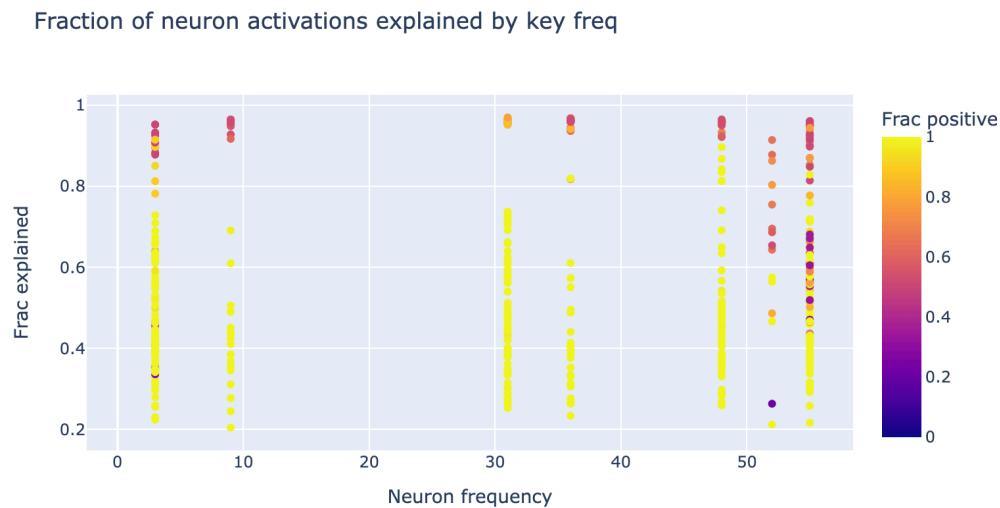


After 5,000 epochs, the model has not yet learned to project into its final key frequencies, whereas after 18,000 epochs the embedding norm closely resembles its final version. Comparing this with the model's training curve (shown below), the Fourier embedding is a qualitative progress measure – the model cannot generalize before it learns to project into key frequencies.

Grokking Training Curve



By calculating the fraction of variance explained by a given frequency for each of the 512 MLP neurons, we find that some neurons fire more strongly to key frequencies and some always fire. Pictured below are the 512 neurons sorted by their key frequency. The y-axis describes the fraction of the neuron's firing variance explained by the key frequency and the color represents the strength of the activation.



This graph provides a couple findings. Firstly, the key frequencies match those from the Fourier embedding (recall that in the embedding diagram sines and cosines are graphed together). This confirms that the MLP neurons favor the same key frequencies and have learned to react to them only. Secondly, while the neurons do respond to the key frequencies, their variance is not always well explained by that frequency alone, represented by their low y-value in the chart. This result is unexpected and differs from those found in Nanda’s original experiment. When a high fraction of a neuron’s variance corresponds to a certain frequency, it can be well approximated with sines and cosines of that frequency. This was proven by replacing each neuron’s activation with its degree-2 polynomial approximation, which led to very small decreases in performance and sometimes improvement on accuracy.

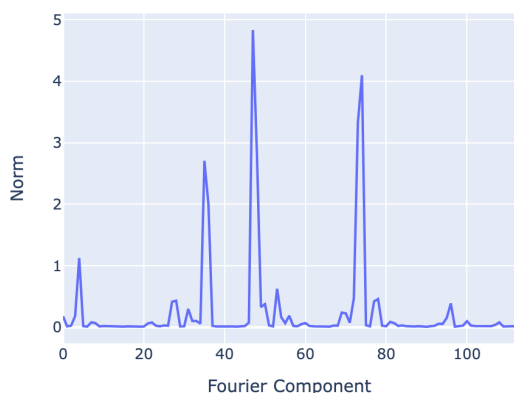
Most of the neurons in this model however are not well approximated by its key frequency and by performing neuron replacement the model’s accuracy is reduced to pure chance.

Original loss	9.67e-8
Loss with neuron activations in key frequency	4.7266
Loss with randomized logits	4.7274

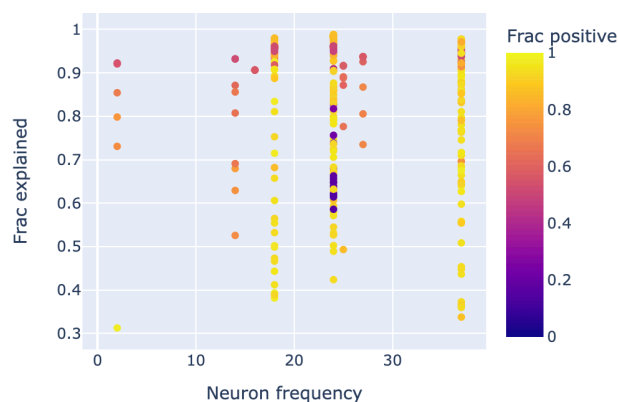
There is some difference in the way my model groks modular addition compared to past experiments, but the data still suggests that the model works using the same algorithm. Lastly, these same frequencies are tracked through the model’s unembedding matrix.

The model trained on modular subtraction could plausibly use the same strategy as in the modular addition case. First project into key frequencies, then compute subtracted sums using trigonometric identities and use constructive interference. This appears to be the case. Below is the Fourier embedding beside the neuron activations. Graphed in this format, it’s possible to see how the frequencies match up.

Norm of embedding of each Fourier Component

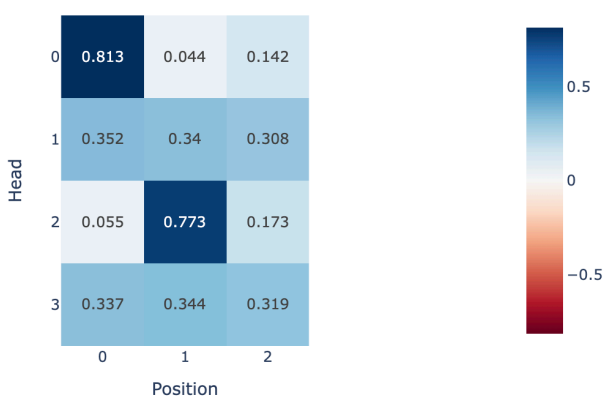


Fraction of neuron activations explained by key freq



If the model is implementing a similar process compared to addition, then why the contrast in training time? One difference between the two model's activations lies in the attention scores for each position. While addition gives equal scores to the first two tokens, subtraction develops a surprising asymmetry.

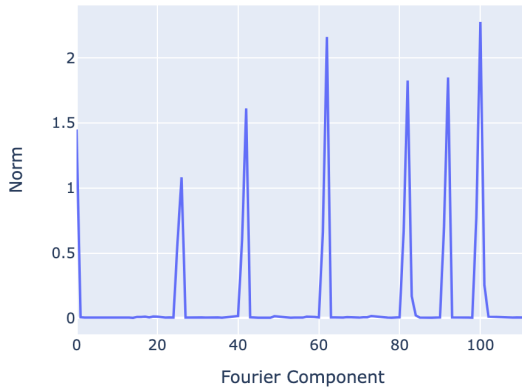
Attention score for heads at each position



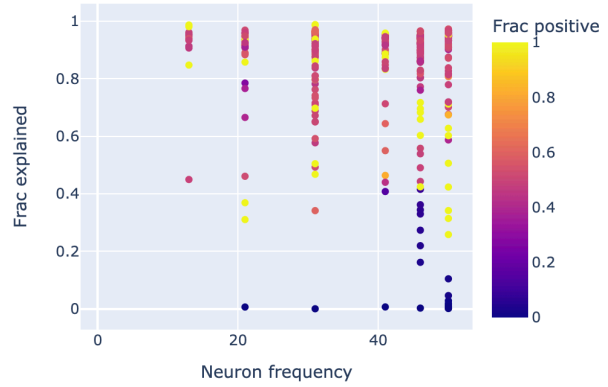
The model devotes two heads to solely give attention scores to one of the two tokens, while the other two heads develop the same equality as in the case of addition. The exact heads which take each role vary between training runs as the model has no reason to favor one over the other. The model is able to account for the non-commutative property of subtraction by developing an asymmetry when processing its input. The remaining features of the model look the same as results from the addition model, so it's difficult to say how exactly its learned algorithm changes for the rest of the process.

The final model trained on joint modular addition and subtraction once again exhibits the same representations in the Fourier space in its activations. In fact, the model's neurons seem to be better described by their key frequencies and the Fourier embedding has less noise.

Norm of embedding of each Fourier Component

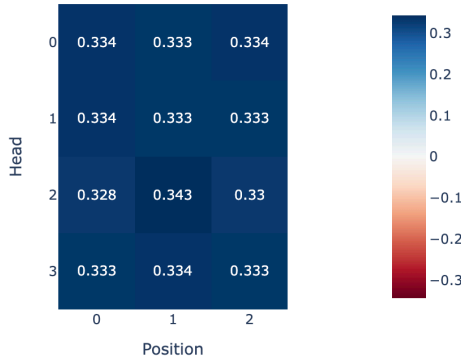


Fraction of neuron activations explained by key freq

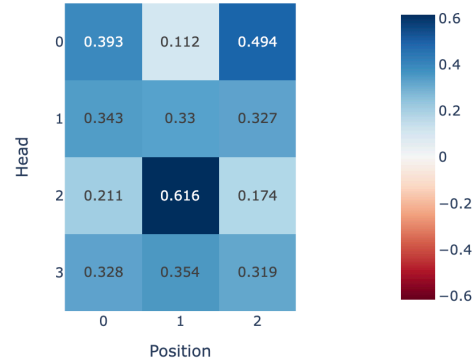


The model is trained to compute modular arithmetic on the first two tokens a and b once again, while the final token is set to $P = 113$ for addition and $P + 1 = 114$ for subtraction. Whereas in previous models where the final token remained constant throughout training, the model must now learn to change its calculation based on the value of the final token. This can be seen through the model's attention activations. Once again, we visualize the average attention scores over all P^2 inputs at each position. On the left, we have the model's scores when the final token is P . On the right, the model calculates attention when the final token is $P+1$.

Attention score for heads at each position



Attention score for heads at each position



When the model recognizes the final token P , its activations are once again symmetrical between the first two tokens. The score is a bit higher on the final token than on the addition model, perhaps because it is now using the final token to differentiate between operations. When the final token is $P+1$, the model once again devotes two heads with asymmetric attention scores and the other two remain symmetrical. In the case of joint arithmetic, it appears that the model develops the same circuits as it did in the individual tasks and uses the final token to switch between the two seamlessly.

6. Conclusion

This experiment began with recreating grokking in transformers on different modular arithmetic tasks. The initial goal was to observe whether the timing of phase changes occurred in predictable ways. The project

later became interested in interpreting the learned algorithms in these transformers as a way of understanding the grokking phenomenon. Sudden generalizations may look random but are well explained by the model slowly forming features that work in key frequencies.

Past work on this topic defines *progress measures* as a quantitative measure of the model's progress. However, my model's MLP neurons were not as well described by trigonometric approximations and I was unable to utilize some of these metrics. This itself is worth noting. Despite my model having key dissimilarities (final LayerNorm, different learning rate), it still exhibits grokking and learns the familiar "clock" algorithm. I also found that the model trained on joint arithmetic demonstrates surprising capability. It was able to learn the addition and subtraction circuits on top of one another, and perform with perfect accuracy on the dataset.

In future work I'd like to look into what the LayerNorm is doing in my model that allows it to grok, as well as tracking exactly what the attention layer does in the joint arithmetic model to move information depending on the operator. Predicting emergent behavior based on problem complexity has proven to be a remaining challenge even on simple tasks, due in part to difficulty in defining problem complexity. Developing theory on this topic could give intuition on how emergent behavior arises in larger models.

Acknowledgments

Thank you to my mentor Carter Teplica, who helped guide me through numerous roadblocks along the way. I benefited greatly during this project through conversations with Zack Rudolph, Oksana Vickers, Max Lamparth, Sanyu Rajakumar, Anastasia Wei, Melody Gui, and Julien Bencheek.

The model used in this project was adapted from Andrej Karpathy's NanoGPT, and the code used for this project can be found here: <https://github.com/Robert-Shin/grokkingxlab>. This work was also made possible by Neel Nanda and his work on finding the "clock" algorithm as well as ARENA education, where I learned how to visualize the model's internals. This project was funded by the UChicago Existential Risk Lab.

As for the author, you can reach me at rshin27@uchicago.edu. I am currently a rising sophomore studying physics at UChicago.

References

- H. Furuta, M. Gouki, Y. Iwasawa, Y. Matsuo, Interpreting grokked transformers in complex modular arithmetic, arXiv preprint arXiv:2402.16726, 2024.
- A. Humayun, R. Balestrieri, R. Baraniuk, Deep networks always grok and here is why. arXiv preprint arXiv:2402.15555, 2024.
- N. Nanda, L. Chan, T. Lieberum, J. Smith, J. Steinhardt, Progress measures for grokking via mechanistic interpretability, in: The Eleventh International Conference on Learning Representations, 2022.
- P. Notsawo Jr, H. Zhou, M. Pezeshki, I. Rish, G. Dumas, Predicting Grokking Long Before it Happens: A look into the loss landscape of models which grok. arXiv preprint arXiv:2306.13253, 2023.

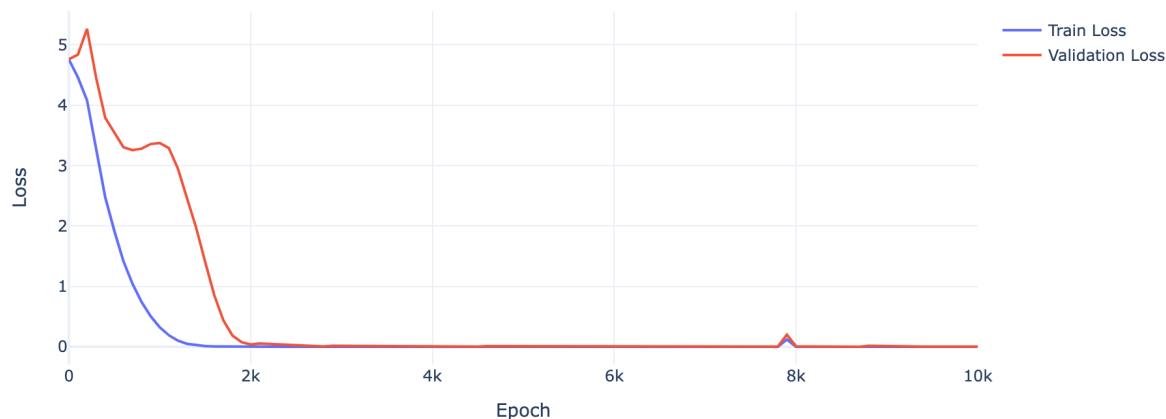
- A. Power, Y. Burda, H. Edwards, I. Babuschkin, V. Misra, Grokking: Generalization beyond overfitting on small algorithmic datasets, arXiv preprint arXiv:2201.02177, 2022.
- B. Wang, X. Yue, Y. Su, H. Sun, Grokked transformers are implicit reasoners: A mechanistic journey to the edge of generalization. arXiv preprint arXiv:2405.15071, 2024.
- J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, et al. Emergent abilities of large language models. arXiv preprint arXiv:2206.07682, 2022.
- Z. Zhong, Z. Liu, M. Tegmark, J. Andreas, The clock and the pizza: Two stories in mechanistic explanation of neural networks. In A. Oh, T. Neumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, Advances in Neural Information Processing Systems, volume 36, pages 27223–27250. Curran Associates, Inc., 2023.

Extra

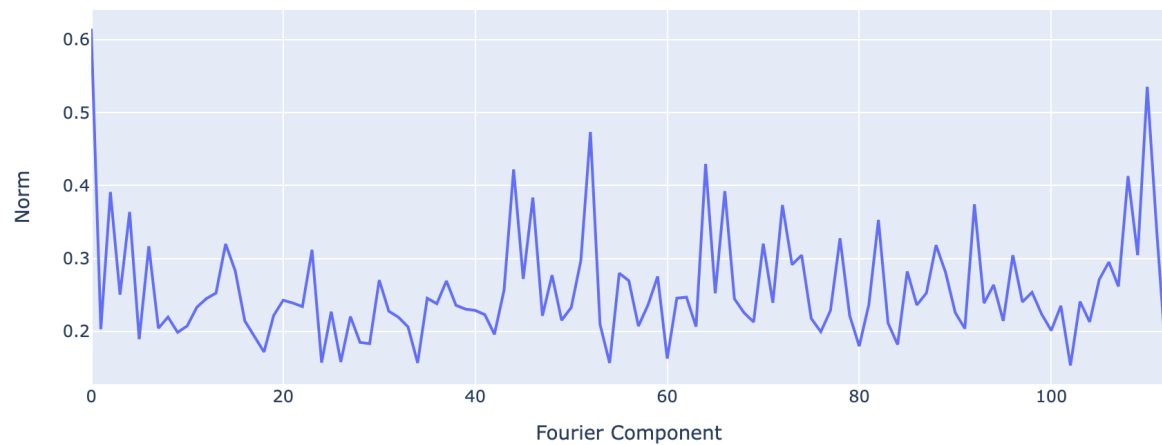
Regarding Modular Multiplication

Below is the loss curve for modular multiplication – note the x-axis has been rescaled to only show 10k epochs. This is because the model learns multiplication much more quickly than addition and subtraction. Despite learning so quickly, the model doesn't reach very low cross-entropy loss (4.3×10^{-4} compared to addition on the order of 10^{-8}) even after 100,000 epochs. Looking at the Fourier embedding, the multiplication algorithm does not seem to be choosing any particular frequencies or to be projecting into sine and cosine at all. Neurons don't appear to activate for any key frequencies either. For this reason, I decided that I would focus on the joint arithmetic model and considered multiplication to be out of scope for this project. Interpreting the modular multiplication model has been done by Furuta et al., and they found that using the pre-grokked embedding from the modular addition model hurt the new model's ability to learn multiplication.

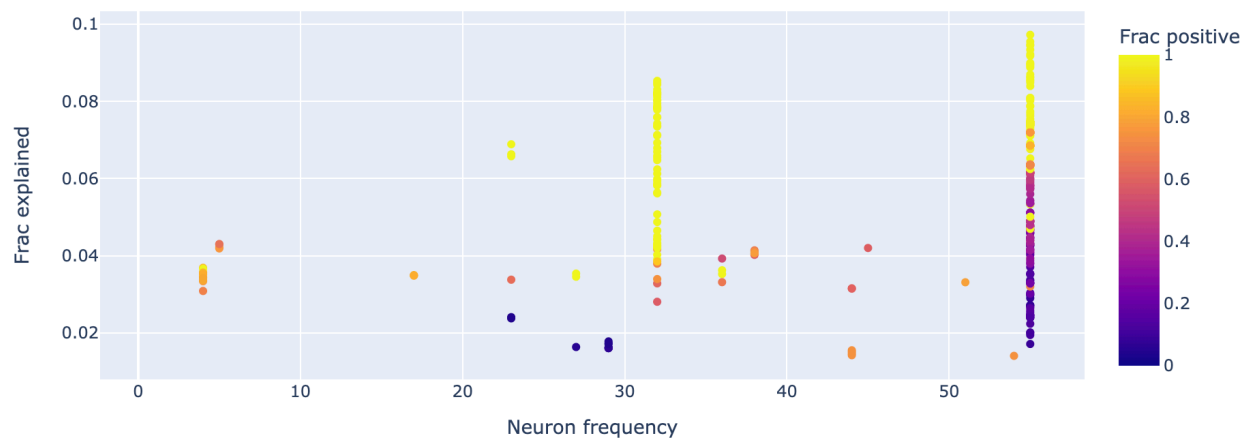
Grokking Training Curve (multiplication)



Norm of embedding of each Fourier Component (multiplication)



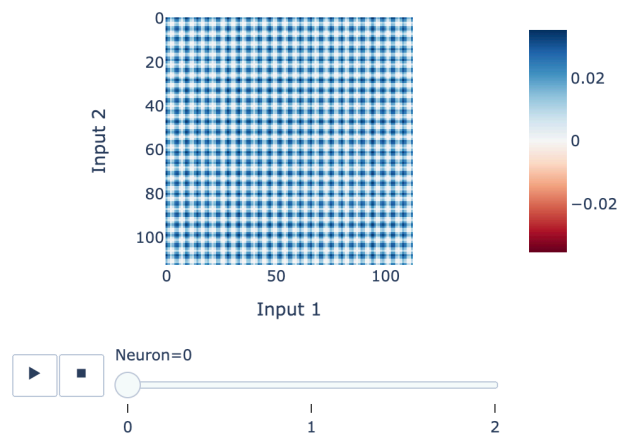
Fraction of neuron activations explained by key freq (multiplication)



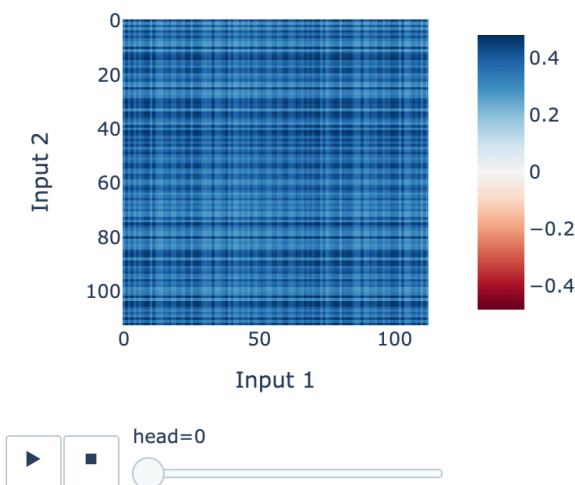
Periodic Activations and Embeddings in the Models

Since they were glossed over in the results section, here are some of the attention+neuron activation visualizations for the models trained on various tasks. Pictured in order are, neuron activation for modular subtraction model, attention score for modular multiplication, and neuron activation for modular multiplication. Note that the second two are not as obviously periodic as past visual representations – the model doesn't project into key frequencies as clearly in the case of multiplication.

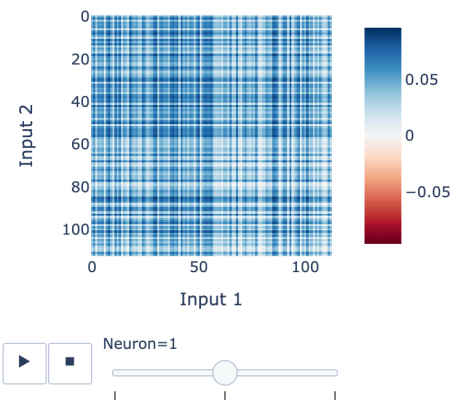
Activations for first 3 neurons



Attention score for heads at position 0



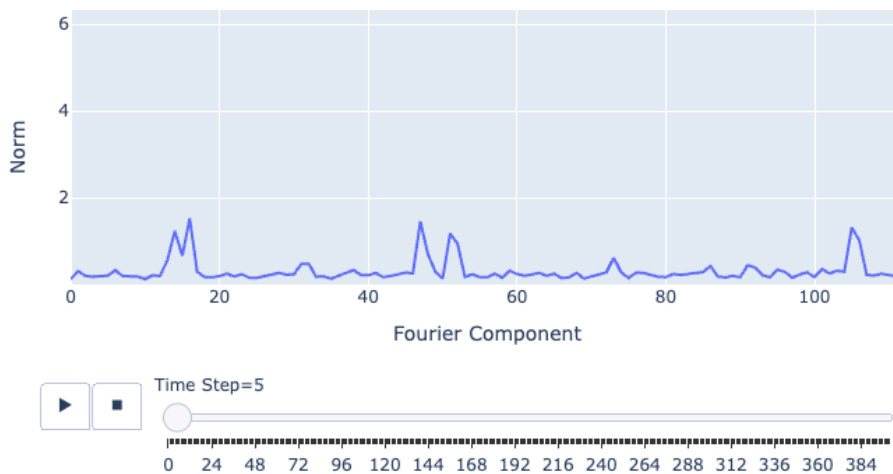
Activations for first 3 neurons



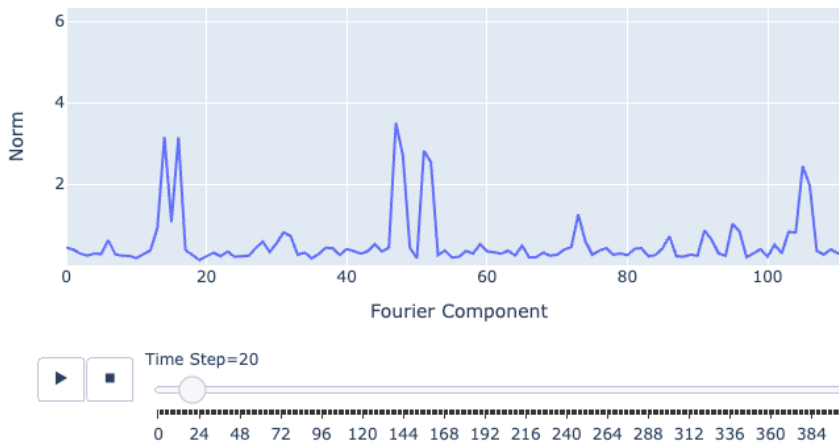
As mentioned in 5.1, seed 1340 appeared to generalize much more quickly than other models grokking modular addition. Below are the Fourier embeddings at 500 epochs, 2,000 epochs and the final embedding

after 40,000 epochs. Compared with the earlier example, this seed seems to already begin forming peaks at its final key frequencies after just 500 epochs. Looking at the loss curve concurrently, the model has already begun to drop in loss by this point. By 2,000 epochs it has already reached near full accuracy. Strangely, the model's lowest cross-entropy loss is 4.1×10^{-5} , around 100 times greater than other training runs. Perhaps the grokking behavior is necessary to reach extremely accurate generalization.

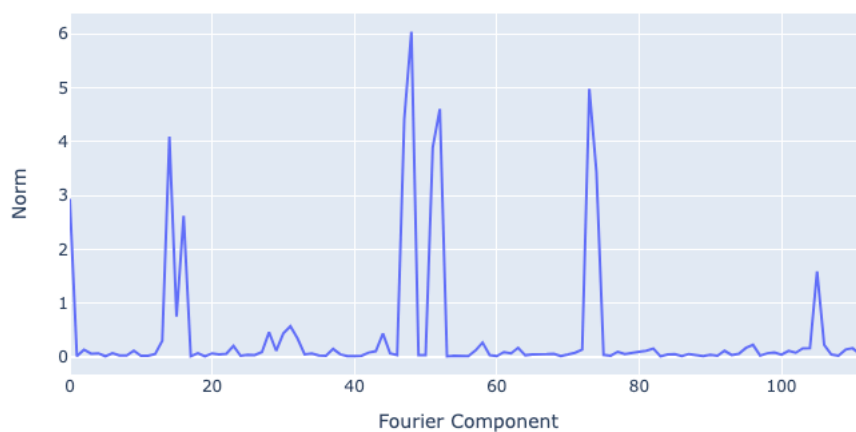
Evolution of Embedding Norm Over Time



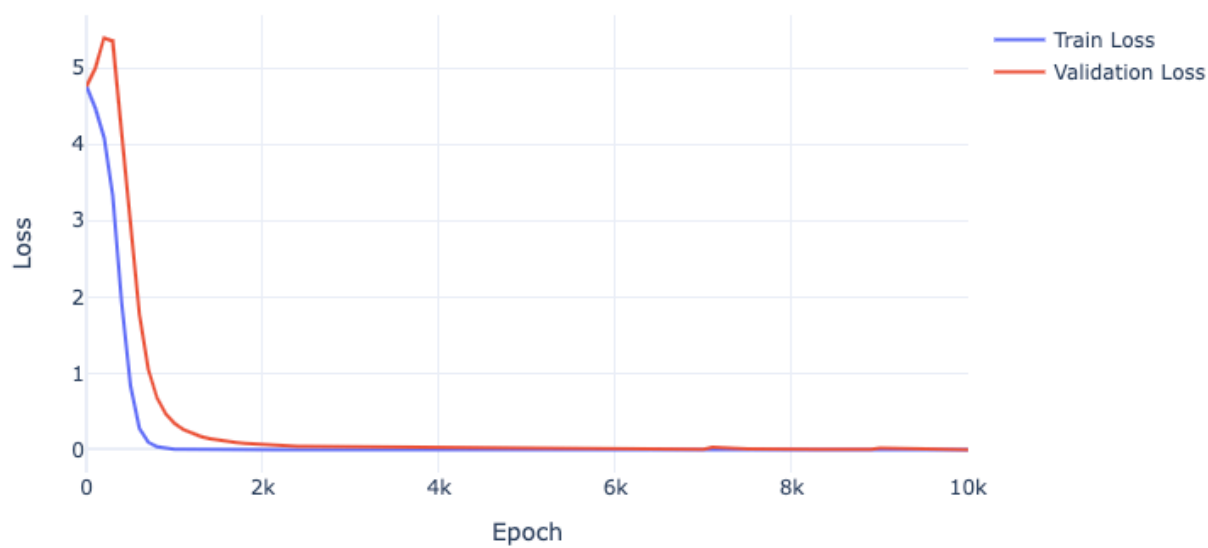
Evolution of Embedding Norm Over Time



Norm of embedding of each Fourier Component



Grokking Training Curve



The unusual training curve of modular addition at seed 1340